

beyond the scope of this MVC are included within target types.

Management UI

Development of an independent interface for managing secrets is out of scope for this blueprint. Any detections will be managed using the existing Vulnerability Management UI.

Management of detected secrets will remain distinct from the Secret Management feature capability as "detected" secrets are categorically distinct from actively "managed" secrets. When a detected secret is identified, it has already been compromised due to their presence in the target object (that is a repository). Alternatively, managed secrets should be stored with stricter standards for secure storage, including encryption and masking when visible (such as job logs or in the UI).

As a long-term priority we should consider unifying the management of the two secret types however that work is out of scope for the current blueprints goals, which remain focused on active detection.

Target types

Target object types refer to the scanning targets prioritized for detection of leaked secrets.

In order of priority this includes:

1. Non-binary Git blobs under 1 megabyte
1. Job logs
1. Container images
1. Creating and updating issues, epics and MRs
1. Comments on issues, epics and MRs

Targets out of scope for now include:

- Non-binary Git blobs over 1 megabyte
- Binary Git blobs
- Media types (JPEG, PDF, ...)
- Snippets
- Wikis
- External media (Youtube platform videos)

Token types

The existing Secret Detection configuration covers 100+ rules across a variety of platforms. To reduce total cost of execution and likelihood of false positives the dedicated service targets only well-defined, low-FP tokens.

Token types to identify in order of importance:

1. Well-defined GitLab tokens (including Personal Access Tokens and Pipeline Trigger Tokens)

1. Verified Partner tokens (including AWS)
1. Well-defined low-FP third party tokens
1. Remainder tokens currently included in Secret Detection analyzer configuration

A well-defined token is a token with a precise definition, most often a fixed substring prefix (or suffix) and fixed length.

For GitLab and partner tokens, we have good domain understanding of our own tokens and by collaborating with partners verified the accuracy of their provided patterns.

An observed low-FP token relies on user reports and dismissal reports. With delivery of this data issue we will have aggregates on FP-rates but primarily this is user-reported data, at present.

In order to minimize false positives, there are no plans to introduce or alert on high-entropy, arbitrary strings; i.e. patterns such as 3lsjkw3a22.

Uniformity of rule configuration

Rule pattern configuration should remain centralized in the secrets analyzer's packaged `gitleaks.toml` configuration, vendored to the monolith for Phase 1, and checksum-checked to ensure it matches the specific release version to avoid drift. Each token can be filtered by tags to form both high-confidence and blocking groupings. For example:

```
ruby
prereceiveblockingrules = toml.loadfile('gitleaks.toml')['rules'].select do |r|
  r.tags.include?('gitlabblockingp1') &&
  r.tags.include?('gitlabblocking')
end
```

Auditability

A critical aspect of both secret detection and suppression is administrative visibility. With each phase we must include audit capabilities (events or logging) to enable event discovery.

Proposal

The first iteration of the experimental capability will feature a blocking pre-receive hook implemented in the Rails application. This iteration will be released in an experimental state to select users and provide opportunity for the team to profile the capability before considering extraction into a dedicated service.

In the future state, to achieve scalable secret detection for a variety of domain objects a dedicated

scanning service must be created and deployed alongside the GitLab distribution. This is referred to as the SecretScanningService.

This service must be:

- highly performant
- horizontally scalable
- generic in domain object scanning capability

Platform-wide secret detection should be enabled by-default on GitLab SaaS as well as self-managed instances.

Decisions

- 001: Use Ruby Push Check approach within monolith
- 002: Store the Secret Detection Gem in the same repository
- 003: Run scan within subprocess
- 004: Standalone Secret Detection Service
- 005: Use Runway for service deployment
- 006: Unified SD Support for all GitLab Environments
- 007: Switch to Vectorscan-based Go scan engine
- 008: Unified SD Scan Engine

Challenges

- Secure authentication to GitLab.com infrastructure
- Performance of scanning against large blobs
- Performance of scanning against volume of domain objects (such as push frequency)
- Queueing of scan requests

Transfer optimizations for large Git data blobs

As described in Gitaly's upload-pack traffic blueprint, we have faced problems in the past handling large data transfers over gRPC. This could be a concern as we expand secret detection to large blob sizes to increase coverage over leaked secrets. We expect to rollout pre-receive scanning with a 1 megabyte blob size limit which should be well within boundaries. From Protobuffers' documentation:

> As a general rule of thumb, if you are dealing in messages larger than a megabyte each, it may be time to consider an alternate strategy.

In expansion phases we must explore chunking or alternative strategies like the optimized sidechannel approach used by Gitaly.

Design and implementation details

The detection capability relies on a multiphase rollout, from an experimental component implemented directly in the monolith to a standalone service capable of scanning text blobs generically.

The implementation of the secret scanning service is highly dependent on the outcomes of our benchmarking and capacity planning against both GitLab.com and Reference Architectures.

As the scanning capability must be an on-by-default component of both our SaaS and self-managed instances, each iteration's deployment characteristic defines whether the service will act as a standalone component, or executed as a subprocess of the Rails architecture (as mirrors the implementation of our Elasticsearch indexing service).

See technical discovery for further background exploration.

See this thread for past discussion around scaling approaches.

Detection engine

Initial Decision

Our current secret detection offering uses Gitleaks for all secret scanning in pipeline contexts. By using its `--no-git` configuration we can scan arbitrary text blobs outside of a repository context and continue to use it for non-pipeline scanning.

Changes to the detection engine are out of scope until benchmarking unveils performance concerns.

For the long-term direction of GitLab Secret Detection, the scope is greater than that of the Gitleaks tool. As such, we should consider feature encapsulation to limit the Gitleaks domain to the relevant build context only.

In the case of pre-receive detection, we rely on a combination of keyword/substring matches for pre-filtering and re2 for regex detections. See spike issue for initial benchmarks.

Notable alternatives include high-performance regex engines such as Hyperscan or its portable fork Vectorscan.

These systems may be worth exploring in the future if our performance characteristics show a need to grow beyond the existing stack, however the team's velocity in building an independently scalable and generic scanning engine was prioritized, see ADR 001 for more on the implementation language considerations.

Recent Consideration

We will use Vectorscan regex engine ported in Go language for the scan engine. See ADR 007 for the reasoning behind the language and engine considerations.

Organization-level Controls

Configuration and workflows should be oriented around Organizations. Detection controls and

governance patterns should support configuration across multiple projects and groups in a uniform way that emphasizes shared allowlists, organization-wide policies (i.e. disablement of push option bypass), and auditability.

Each phase documents the paradigm used as we iterate from Instance-level to Organization-level controls.

Phase 1 - Ruby pushcheck pre-receive integration

The critical paths as outlined under goals above cover two major object types: Git text blobs (corresponding to push events) and arbitrary text blobs. In Phase 1, we focus entirely on Git text blobs.

The detection flow for push events relies on subscribing to the PreReceive hook to scan commit data using the PushCheck interface. This SecretScanningService service fetches the specified blob contents from Gitaly, scans the commit contents, and rejects the push when a secret is detected. See Push event detection flow for sequence.

In the case of a push detection, the commit is rejected inline and error returned to the end user.

Configuration

This phase will be considered "experimental" with limited availability for customer opt-in, through instance level application settings.

High-Level Architecture

The Phase 1 architecture involves no additional components and is entirely encapsulated in the Rails application server. This provides a rapid deployment with tight integration within auth boundaries and no distribution coordination.

The primary drawback relies on resource utilization, adding additional CPU, memory, transfer volume, and request latency to existing application nodes.

plantuml

@startuml Phase2

skinparam linetype ortho

card "External Load Balancer" as elb 6a9be7

together {

card "GitLab Rails" as gitlab 32CD32

card "Gitaly" as gitaly FF8C00

card "PostgreSQL" as postgres 4EA7FF

card "Redis" as redis FF6347

card "Sidekiq" as sidekiq ff8dd1

}

}

```
gitlab -[32CD32]--> gitaly
gitlab -[32CD32]--> postgres
gitlab -[32CD32]--> redis
gitlab -[32CD32]--> sidekiq
```

```
elb -[6a9be7]-> gitlab
```

```
gitlab .[32CD32]----> postgres
sidekiq .[ff8dd1]----> postgres
```

@enduml

Push Event Detection Flow

```
mermaid
sequenceDiagram
    autonumber
    actor User
    User->>+Workhorse: git push with-secret
    Workhorse->>+Gitaly: tcp
    Gitaly->>+Rails: PreReceive
    Rails->>-Gitaly: ListAllBlobs
    Gitaly->>-Rails: ListAllBlobsResponse

    Rails->>+GitLabSecretDetection: Scan(blob)
    GitLabSecretDetection->>-Rails: found

    Rails->>User: rejected: secret found

    User->>+Workhorse: git push without-secret
    Workhorse->>+Gitaly: tcp
    Gitaly->>+Rails: PreReceive
    Rails->>-Gitaly: ListAllBlobs
    Gitaly->>-Rails: ListAllBlobsResponse

    Rails->>+GitLabSecretDetection: Scan(blob)
    GitLabSecretDetection->>-Rails: notfound

    Rails->>User: accepted
```

Gem Scanning Interface

For the Phase1, we use the private Secret Detection Ruby Gem that is invoked by the Secrets Push Check on the GitLab Rails platform.

The private SD gem offers the following support in addition to running scan on multiple blobs:

- Configurable Timeout on the entire scan-level and on each blob level.

- Ability to run the scan within subprocess instead of the main process. The number of processes spawned per request is capped to 5.

The Ruleset file referred during the Pre-receive Secret Detection scan is located here.

More details about the Gem can be found in the README file. Also see ADR 002 for more on how the Gem code is stored and distributed.

Phase 2 - Standalone Secret Detection service

This phase emphasizes scaling the service outside of the monolith for general availability, isolating feature's resource consumption, and ease of maintainability. The critical paths as outlined under goals above cover two major object types: Git text blobs (corresponding to push events) and arbitrary text blobs. In Phase 2, we continue to focus on Git text blobs.

The responsibility of the service will be limited to running Secret Detection scan on the given set of input blobs. More details about the service are outlined in ADR 004: Secret Detection Scanner Service.

The introduction of a dedicated service impacts the workflow for Secret Push Protection as follows:

```
mermaid
sequenceDiagram
    autonumber
    %% Phase 2: Iter 1
    Gitaly->>+Rails: invokes /internal/allowed API endpoint
    Rails->>Rails: Perform project eligibility checks
    alt On access check failure
        Rails-->>Gitaly: Scanning Skipped
    end
    Rails->>Gitaly: Fetch blobs
    Gitaly->>Rails: Quarantined Blobs
    Rails->>Secret Detection Service: Invoke scan by embedding blobs
    Secret Detection Service->>Secret Detection Service: Runs Secret Detection on input blobs
    Secret Detection Service->>Rails: Result
    Rails->>Gitaly: Result
```

The Secret Detection service addresses the previous phase's limitations of feature scalability and shared-resource consumption. However, the Secret Push Protection workflow still requires Rails monolith to load large amount of Git blobs fetched from Gitaly into its own memory before passing it down to the Secret Detection Service.

Phase 2.1 - Invoke Push Protection directly from Gitaly

Until the previous phase, there are multiple hops made between Gitaly and Rails for running Pre-receive checks, particularly for Secret Push protection so a fairly large amount of Rails memory is occupied for holding Git blobs to pass them to the Gem/Service for running secret scan. This problem can be mitigated through a direct interaction between the Secret Detection service and Gitaly via standard interface (either Custom pre-receive hook or Gitaly's new Plugin-based architecture). This setup skips the need for Rails to be a blob messenger between Gitaly and Service.

Gitaly's new Plugin-based architecture is the preferred interface for interacting b/w Gitaly and RPC service as it provides streamlined access to the Git blob repository. However, Gitaly team is yet to take it up for development.

More details on Phase 2.1 will be added once there are updates on the development of Plugin architecture.

Phase 3 - Expansion beyond Push Protection service

The standalone Secret Detection service is currently supported for GitLab.com environment. We will continue using a hybrid approach of SD Service (GitLab.com) and Embedded Secret Detection module(Self-Managed/Dedicated). An embedded Secret Detection module is a locally hosted Secret Detection application in the form of a Ruby gem or an executable binary installed on the host GitLab Rails machine.

Secret Push Protection requires scans to run on git diffs (target type) in a blocking manner to provide scan results instantaneously. This approach isn't technically scalable for other scan target types (particularly larger sized ones like Job Artifacts or Job Logs). As a result, we decided to support non-blocking scans via Sidekiq.

[Read more about the above decisions here.](#)

High-level for Unified SD Scan Engine

A single scanning engine for all scan target types and with the help of target type-specific Adapters, the overall design of Secret Detection for various scan target types looks like the following illustration:

[Read more details about unified scan engine here.](#)

High-level SD detection flow for Work Items

The detection flow for arbitrary text blobs, such as issue comments, relies on

subscribing to `Notes::PostProcessService` (or equivalent service) to enqueue Sidekiq requests to the `SecretScanningService` to process the text blob by object type and primary key of domain object. The `SecretScanningService` service fetches the relevant text blob, scans the contents, and notifies the Rails application when a secret is detected.

High-level SD detection flow for Job Logs

The detection flow for job logs requires processing the log during archive to object storage. See discussion in this issue around scanning during streaming and the added complexity in buffering lookbacks for arbitrary trace chunks.

Configuration

This phase will be considered "generally available" and on-by-default, with disablement configuration through organization-level settings.

High-Level Architecture

There is no change to the architecture defined in Phase 2, however the individual load requirements may require scaling up the node counts for the detection service.

Push Event Detection Flow

There is no change to the push event detection flow defined in Phase 2, however the added capability to scan arbitrary text blobs directly from Rails allows us to emulate a pre-receive behavior for issuable creations, as well (see target types for priority object types).

```
mermaid
sequenceDiagram
    autonumber
    actor User
    User->>+Workhorse: git push with-secret
    Workhorse->>+Gitaly: tcp
    Gitaly->>+GitLabSecretDetection: PreReceive
    GitLabSecretDetection->>-Gitaly: ListAllBlobs
    Gitaly->>-GitLabSecretDetection: ListAllBlobsResponse

    Gitaly->>+GitLabSecretDetection: PreReceive

    GitLabSecretDetection->>GitLabSecretDetection: Scan(blob)
    GitLabSecretDetection->>-Gitaly: found

    Gitaly->>+Rails: PreReceive

    Rails->>User: rejected: secret found
```

User->>+Workhorse: POST issuable with-secret

Workhorse->>+Rails: tcp

Rails->>+GitLabSecretDetection: PreReceive

GitLabSecretDetection->>GitLabSecretDetection: Scan(blob)

GitLabSecretDetection->>-Rails: found

Rails->>User: rejected: secret found

Future Phases

These are key items for delivering a feature-complete always-on experience but have not yet been prioritized into phases.

Large blob sizes (1mb+)

Current phases do not include expansions of blob sizes beyond 1mb. While the main limitation was chosen to conform to RPC transfer limits for future iterations we should expand to supporting additional blob sizes. This can be achieved in two ways:

1. Post-receive processing

Accept blobs in a non-blocking fashion, process scanning as background job and alert passively on detection of a given secret.

1. Improvements to scanning logic batching

Maintaining the constraint of 1MB is primarily futureproofing to match an expected transport protocol. This can be mitigated by using separate transport (http, reads from disk, ...) or by slicing blob sizes.

Detection Suppression

Suppression of detection and action on leaked secrets will be supported at several levels.

1. Global suppression - If a secret is highly-likely to be a false token (i.e. EXAMPLE) it should be suppressed in workflow contexts where user would be seriously inconvenienced.

We should still provide some means of triaging these results, whether via audit events or as automatic vulnerability resolution.

1. Organization suppression - If a secret matches an organization's allowlist (or was previously flagged and remediated as irrelevant) it should not reoccur. See Organization-level controls.

1. Inline suppression - Inline annotations should be supported in later phases with the Organization-level configuration to ignore annotations.

External Token Verification

As a post-processing step for detection we should explore verification of detected secrets. This requires processors per supported token type in which we can distinguish tokens that are valid leaks from false positives. Similar to our automatic response to leaked secrets, we must externally verify a given token to give a high degree of confidence in our alerting.

There are two token types: internal and external:

- Internal tokens are verifiable and revocable as part of ScanSecurityReportSecretsWorker worker
- External tokens require external verification, in which the architecture will closely match the Secret Revocation Service

Iterations

- · Define requirements for detection coverage and actions
- · Implement Browser-based detection of GitLab tokens in comments/issues
- · PoC of secret scanning service
- · PoC of secret scanning gem
- Pre-Production Performance Profiling for pre-receive PoCs
 - Profiling service capabilities
 - · Benchmarking regex performance between Ruby and Go approaches
 - transfer latency, CPU, and memory footprint
- · Implementation of secret scanning gem integration MVC (targeting individual commits)
- Phase1 - Deployment and monitoring
- Capacity planning for addition of service component to Reference Architectures headroom
- Security and readiness review
- Phase2 - Deployment and monitoring
- Implementation of secret scanning service (targeting arbitrary text blobs)
- Phase3 - Deployment and monitoring
- High priority domain object rollout (priority TBD)
 - Issuable comments
 - Issuable bodies
 - Job logs

SECTION: engineering/architecture/design-
documents/secret_detection/decisions/001_use_ruby_push_check_approach_within_monolith.md

Context

There are a number of concerns around the performance of secret detection using a regex-based approach at scale. The primary considerations include transfer latency between nodes and both CPU and memory bloat. These concerns manifested in two ways: the language to be used for performing regex matching and the deployment architecture.

The original discussion in the exploration issue covers many of these concerns and background.

Implementation language

The two primary languages considered were Ruby and Go.

The choice to use other languages (such as C++) for implementation was discarded in favour of Ruby and Go due to team familiarity, speed of deployment, and portability. See this [benchmarking issue](#) for performance comparisons between the two.

Deployment architecture

Several options were considered for deployments: directly embedding the logic within the Rails monolith's Push Check execution path, placement as a sidecar within a Rails node deployment, placement as a sidecar within a Gitaly node as a server-side hook, and deployment as a standalone service.

Decision

For the initial iteration around blocking push events using a prereceive integration, the decision was made to proceed with Ruby-based approach, leveraging re2 for performant regex processing. Additionally, the decision was made to integrate the logic directly into the monolith rather than as a discrete service or server-side hook within Gitaly.

A Gitaly server-side hook would have performance benefits around minimal transfer latency for Git blobs between scanning service and Gitaly blob storage. However, an extra request would be needed between Gitaly and the Rails application to contextualize the scan. Additionally, the current hook architecture is discouraged and work is planned to migrate towards a new plugin architecture in the near future.

The Ruby Push Check approach follows a clear execution plan to achieve delivery by anticipated timeline and is more closely aligned with the long-term direction of platform-wide scanning. For example, future scanning of issuables will require execution within the trust boundary of the Rails application rather than Gitaly context. This approach, however, has raised concerns around elevated memory usage within the Rails application leading to availability concerns. This direction may also require migrating towards Gitaly's new plugin architecture in the future once the timeline is known.

A standalone service may be considered in the future but requires considerations of a technical approach that should be better informed by data gathered during pre-production profiling.

SECTION: engineering/architecture/design-documents/secret_detection/decisions/002_store_the_secret_detection_gem_in_the_same_repository.md

Context

During Phase 1, we opted for using the Ruby-based push check approach to block secrets from being committed to a repository, and as such the scanning of secrets was performed by a library (or a Ruby gem) developed internally within GitLab for this specific purpose.

Part of the process to create this library and make it available for use within the Rails monolith, we had to make a decision on the best way to distribute the library.

Approach

We evaluated two possible approaches:

1. Store the library in the same repository as the monolith.
1. Store the library in an external repository.

Each approach came with some advantages and disadvantages, mostly around distribution, consistency, maintainability, and the overhead of having to set up review and release workflows and similar processes. See below for more information.

Within the same repository as the monolith

Having the gem developed and stored in the same repository meant having it packaged within GitLab monolith itself, and with that ensuring it does not have to be installed as a dependency. This would also reduce maintainability overhead in terms of defining workflows and processes from scratch. On the other hand, the library would have less visibility as it is not exposed or published to the wider community.

In an external repository

Storing the library in an external repository meant having more visibility especially as the gem would be published on RubyGems.org, which would have garnered more interest and possibly contributions from the community into the feature. Additionally, the gem would be available to be used in other projects and applications. However, in doing so, the maintainability overhead would have increased significantly for various reasons such as:

- Changes would need to be coordinated between multiple repositories when a new version is released.
- Review and release workflows, and similar processes would need to be defined separately.

Decision

The decision was made to store the library in the same repository during the first phase to ensure easier distribution since it's packaged within GitLab and will be available immediately without having to install external dependencies.

With that said, we still followed the process to reserve the gem on RubyGems.org to avoid name-squatters from taking over the name and providing malicious code to 3rd-parties.

We have no plans to publish the gem externally at least until Phase 2 as we begin to consider building a standalone service to perform secret detection.

SECTION: engineering/architecture/design-
documents/secret_detection/decisions/003_run_scan_within_subprocess.md

Context

During the spike conducted for evaluating regex for Pre-receive Secret Detection, Ruby using RE2 library came out on the top of the list. Although Ruby has an acceptable regex performance, its language limitations have certain pitfalls like more memory consumption and lack of parallelism despite the language supporting multi-threading and Ractors (3.1+) as they are suitable for running I/O-bound operations in parallel but not CPU-bound operations.

One of the concerns running the Pre-receive Secret Detection feature in the critical path is memory consumption, especially by the regex operations involved in the scan. In a scan with 300+ regex-based rule patterns running on every line of the commit blobs, the memory could go up to ~2-3x the size of the commit blobs¹. The occupied memory is not released despite scan operation being complete, until the Garbage Collector triggers. Eventually, the servers might choke on the memory.

The original discussion issue covers many of these concerns and more background.

Approach

We can tackle the memory consumption problem to an extent by running the scan within a separate process forked from the main process. Once the scan is complete, we kill the spawned process such that the occupied memory releases to the OS immediately instead of waiting for Ruby to trigger GC.

Technical Solution

There are several scenarios to consider while managing a process's lifecycle. Failing to do so would lead to an orphan process having no control over it, defeating the whole purpose of conserving memory. We offload this burden over a Ruby library called Parallel that provides the ability to run operations via subprocesses. Its simple interface for communication b/w parent and child processes, handling exit signals, and easy capping of no. of processes makes it a suitable solution for achieving our needs. It additionally supports parallelism (spawning and running multiple subprocesses simultaneously) that solves another problem not covered in this document.

Scope of the operation within Subprocess

It is crucial to determine which operation runs within the subprocess because spawning a new process comes with an additional latency overhead from the OS (copying file descriptors, etc). For example, running the scan on each blob inside a new subprocess is ~2.5x slower than when the scan runs on the main process. On the contrary, dedicating one subprocess for each commit request isn't feasible either as the scan on all the blobs runs within a single process and we wouldn't be able to release memory quickly until all the scans are complete, taking us back to square one.

Bucket Approach: A compromise between the two extremes would be when we group all the blobs whose cumulative size is at least a fixed chunk size (2MiB in our case) and then run each group within a separate sub-process as illustrated below.

Addendum

- Running operations within a subprocess isn't a silver bullet to the above mentioned problems. We could say it delays our servers from getting choked by releasing the memory faster than the usual process via GC. Even this approach can fail when the burst of requests is too huge to handle^.
- There's always a latency overhead on the process creation of the lifecycle. For the smaller commits^, the latency of the scan operation might be slower than when run on the main process.
- The parallelism factor or the no. of processes forked per request is currently capped to 5 processes, beyond which pending requests wait in the queue to avoid over-forking processes which would also lead to resource exhaustion.

^Threshold numbers will be added here soon for reference.

SECTION: engineering/architecture/design-
documents/secret_detection/decisions/004_secret_detection_scanner_service.md

Context

In the phase 2 of Secret Push Protection, the goal is to have a dedicated service responsible for running Secret Detection scans on the given input blobs. This is done primarily from the scalability standpoint. Regex operations in the Secret Detection scan consume high resources so running scans within Rails or Gitaly instances would impact the resource availability for running other operations. Running scans in isolation provides greater control over resource allocation and scaling the service independently as needed.

Proposed Solution

We will build a standalone Secret Detection service responsible for running the Secret Detection scans.

The main change in the workflow of Secret Push Protection would be the delegation of scanning responsibility from the Secret Detection gem to the RPC service for GitLab SaaS i.e., the secrets push check invokes the RPC service with an array of blobs to scan for secrets. Note that the project eligibility checks are still performed at the Rails side.

High-Level Architecture

The service architecture involves extracting the secret detection logic into a standalone service which communicates directly with both the Rails application and Gitaly. This provides a means to scale

the secret detection nodes independently, and reduce resource usage overhead on the Rails application.

Scans still runs synchronously as a (potentially) blocking pre-receive transaction. The blob size remains limited to 1MB.

Note that the node count is purely illustrative, but serves to emphasize the independent scaling requirements for the scanning service.

plantuml

@startuml Phase2
skinparam linetype ortho

card "External Load Balancer" as elb 6a9be7
card "Internal Load Balancer" as ilb 9370DB

together {
 collections "GitLab Rails x3" as gitlab 32CD32
 collections "Sidekiq x3" as sidekiq ff8dd1
}

together {
 collections "Consul x3" as consul e76a9b
}

card "SecretScanningService Cluster" as prsdcluster {
 collections "SecretScanningService x5" as prsd FF8C00
}

card "Gitaly Cluster" as gitalycluster {
 collections "Gitaly x3" as gitaly FF8C00
}

card "Database" as database {
 collections "PGBouncer x3" as pgbouncer 4EA7FF
}

elb -[6a9be7]-> gitlab

gitlab -[32CD32,norank]--> ilb
gitlab .[32CD32]----> database
gitlab -[hidden]-> consul

sidekiq -[ff8dd1,norank]--> ilb
sidekiq .[ff8dd1]----> database
sidekiq -[hidden]-> consul

ilb -[9370DB]--> prsdcluster

ilb -[9370DB]--> gitalycluster
ilb -[9370DB]--> database
ilb -[hidden]u-> consul

consul .[e76a9b]u-> gitlab
consul .[e76a9b]u-> sidekiq
consul .[e76a9b]-> database
consul .[e76a9b]-> gitalycluster
consul .[e76a9b]-> prsdcluster

@enduml

Service Level Indicators(SLIs)

We will adopt the same SLIs followed for GitLab Applications i.e.,
Apdex score, Error Ratio and two additional metrics specific to the service - Request Latency
and
Memory Saturation rate.

Service Level Objectives(SLOs)

We will define threshold limits after obtaining benchmark scores from the RPC service.

Service Implementation

We will build an RPC service primarily responsible for detecting secrets in the given input
blobs with RPC as the
communication interface. This service will initially be invoked by Rails monolith when
performing change access checks
for Git Push event, and eventually extended for the other use cases too.

To reuse the same business logic of Scanning, in addition to offering the feature as an RPC
service, the same project
will also include the provision for distributing the feature into a Ruby Gem.

Language/Tools/Framework

- Ruby 3.2+
- gRPC framework for serving RPC requests
- Protobuf Service Definition file

Addendum

- The RPC service should also expose Health Check RPC endpoint for ensuring the availability of the service.
- Unlike in Gem-based approach, we wouldn't be able to use scan within the subprocess approach in the RPC server since they removed support for forking subprocess. However, we can explore optimizations like

batch requests
concurrently from the RPC client side.

Reference links

- Standalone Service as a concept
- Runway: Service Deployment & Docs

SECTION: engineering/architecture/design-
documents/secret_detection/decisions/005_use_runway_for_deployment.md

Context

The Secret Detection Service requires a strategy for running automated deployments via GitLab CI environment.

Proposed Solution: Runway

We could use Runway - a GitLab internal Platform as a Service, which aims to enable teams to deploy and run their services quickly and safely.

Platform Tooling Support

- Logging: Logging on GitLab-managed Elasticsearch/Kibana stack isn't available in Runway and there doesn't seem to be plans to support it anytime soon. At the moment, the workaround is to view logs on Google Cloud Run UI.
- Observability: Runway supports observability for service by integrating with the monitoring stack. The default metrics(example dashboard) provided by Runway covers all the necessary system metrics for monitoring.
- Pager alerts on failures: Runway generates alerts for the following anomalies by default, which we believe are sufficient to get started with:
 - Apdex SLO violation
 - Error SLO violation
 - Traffic absent SLO violation
- Service Level Indicators (SLIs): The default metrics(example dashboard) provided by Runway covers necessary SLI requirements.
- Insights: We might need additional metrics on rule patterns like their latency, usage count, source, etc. We may

use custom metrics, which we will evaluate further soon.

Known Limitations (relevant to Secret Detection Service)

- ~~No support for GRPC protocol~~ Update: GRPC is now supported
- No support for GitLab Self-Managed environments (Reference)

Working with Limitations

The limitation of Runway's missing support for Self-Managed(SM) environments made us evaluate other solutions for SM environments. The Cloud Connector's API-based approach would generally address the missing deployment solution for SM environments. However, the Secret Push Protection feature involves frequent transferring large amounts of data between Gitaly and Service in real-time so REST-based APIs aren't the right fit as they'd add significant network overhead unlike streaming data on an RPC request. We could optimize the Cloud Connector approach with some additional complexity but it will be a matter of time until Runway introduces a deployment solution for SM environments. One more alternative solution for SM environments was to share the Docker image artifact along with deployment instructions with the customers (similar to custom models approach) but the horizontal scaling could be a concern.

We came up with a hybrid solution. To address the scale of GitLab SaaS, we will have a dedicated RPC-based Secret Detection service deployed using Runway. This service will isolate the SD resource usage without impacting the resources of other services(Rails and Gitaly) and can scale independently as needed. Whereas for Self-Managed instances, we will continue using the current gem-based approach since that approach performed adequately for up to GET 50K Reference architecture. We will eventually migrate Self-Managed environments to Runway when it introduces the deployment support.

TL;DR: We will use RPC service (deployed using Runway) for GitLab SaaS, and continue using the current Ruby gem approach for GitLab Self-Managed instances.

To reuse the core implementation of Secret Scanning, we will have a single source code with two different distributions:

1. Wrap a Ruby gem around the secret detection logic and use it in the Rails(replacing the current gem).

1. Wrap an RPC service around the secret detection logic, deploy it using Runway, and invoke the service from Rails for GitLab SaaS

{width="1001" height="311"}

Here's the workflow illustrating the proposed change:

```
mermaid
sequenceDiagram
    autonumber
    %% Phase 2: Iter 1
    Gitaly->>+Rails: invokes internal/allowed
    Rails->>Rails: Perform project eligibility checks
    alt On project eligibility check failure
        Rails-->>Gitaly: Scanning Skipped
    end
    Rails->>Gitaly: Get blobs
    Gitaly->>Rails: Quarantined Blobs
    Note over Rails,SD Ruby Gem: For GitLab Self-Managed
    Rails->>SD Ruby Gem: Invoke RPC and forward quarantined blobs
    SD Ruby Gem->>SD Ruby Gem: Runs Secret Detection on input blobs
    SD Ruby Gem->>Rails: Result
    Note over Rails,SD RPC Service: For GitLab SaaS (GitLab.com & Dedicated)
    Rails->>SD RPC Service: Invoke RPC and forward quarantined blobs
    SD RPC Service->>SD RPC Service: Runs Secret Detection on input blobs
    SD RPC Service->>Rails: Result
    Rails->>Gitaly: Result
```

Reference Links

- Runway Docs
- Epic: Runway - Platform tooling to support AI Innovation
- Blueprint: GitLab Service-Integration: AI and Beyond

SECTION: engineering/architecture/design-
documents/secret_detection/decisions/006_support_for_all_environments.md

Context

We need to take steps in making Secret Detection features scalable and consistently accessible across GitLab.com, Self-Managed (including air-gapped) and Dedicated environments.

There are two problems that we should address, preferably both at the same time to avoid the expensive redesigns.

1. The standalone Secret Detection Service (deployed via Runway) is currently not accessible by GitLab Dedicated and Self-Managed(SM) environments (because Runway isn't supporting in those environments yet). GitLab Cloud Connector was considered for a brief moment but discarded due to certain limitations.

2. The current design of Rails directly invoking the Secret Scan engine(Gem/Secret Detection Service(SDS)) supports only blocking scan requests. However, it is not scalable to do run blocking scans on large objects like Job Artifacts or Job Logs as it affects the throughput of the Scan engine. We need to adopt a non-blocking approach where the scans are run in the background and provide results eventually, similar to how we generate and ingest Security Reports.

Proposal

Multiple proposals were discussed for the above problems, this ADR drafts a combined approach considering relevant proposals.

The proposal suggests addressing the first problem by running SD scans within an Embedded SD module (Gem/Binary) for Self-Managed (including offline) and Dedicated environments. This is a default setup that should be capable enough to handle the traffic for a single-tenant environment. As an alternative, we allow the customers to self-host Secret Detection Service in their infrastructure when the Embedded SD module's throughput isn't meeting their needs. When a self-hosted service URL is provided, we will prefer invoking such a service over the Embedded SD module.

The second problem is addressed by introducing an asynchronous way of invoking secret detection scans using the existing Sidekiq infrastructure.

Support Matrix

We will continue using SDS for GitLab.com and the embedded approach (i.e Gem/Binary) for Self-Managed and Dedicated customers as a default setup, however, we will allow customers to self-host SDS in case the embedded approach is not scalable enough for their use case.

We will leverage the existing background processing tool (Sidekiq) to run non-blocking SD scans in the background. The scan engine remains the same as the one used for blocking approach.

Environment	Blocking scan requests	Non-blocking scan requests
GitLab.com	Runway-hosted SDS	Sidekiq + Runway-hosted
Self-Managed/Dedicated (default)	Embedded(Gem/Binary)	Sidekiq + Embedded
Self-Managed/Dedicated (custom)	Self-hosted SDS	Sidekiq + Self-hosted SDS

Provision for Self-Hosting Service

We could accomplish this in two ways:

1. Share the SDS Docker image with the customers. Let them self-deploy it in their own infrastructure and update the deployed SDS's host URL in GitLab Application Settings. If the URL is defined, we will attempt calling the host URL over embedded SD module. Secret Revocation Service follows this approach.
2. Share the Helm Chart for deploying SDS. This will reduce operational burden of managing the service for the customer. However, this approach is suitable only for the customers running their infrastructure in Kubernetes.

First approach seems simple enough to get started with, and we could eventually provide such a Helm Chart if there is an ask from the customer side.

Service Authentication and Authorization

To ensure the authenticity of requests to the service (primarily applicable to Self-hosted), we could adopt Auth framework from Cloud Connector.

NOTE: This decision still requires evaluation in terms of feasibility.

SECTION: engineering/architecture/design-
documents/secret_detection/decisions/007_switch_to_go_scan_engine.md

Context

This ADR captures the decision to switch from RE2-based(port) Ruby scan engine to Vectorscan-based(port) Go scan engine for Secret Detection scans.

Here's the historical context on why Ruby was chosen and the problems associated with it.

Why Ruby was chosen earlier?

In the initial days of Secret Push Protection(SPP) feature development, we chose Ruby for running SD scans for two main reasons:

1. The expected team's velocity to build the SPP feature out-of-box was pretty high and adding tight time constraints in to the mix led us choosing an approach that had least resistance without compromising much on the quality. Given the team's familiarity with Ruby and ease of integration with GitLab Rails, building the SPP feature within a Ruby Gem made sense.
2. Using RE2 engine ported in Ruby boosted our confidence in the decision as it performed much better in terms latency and memory consumption than the other language+regex engine in the bucket i.e., Go with built-in regex engine.

Fast-forward to the technical discovery phase of building Secret Detection Service(SDS). We retained the decision of using the same language(Ruby) and engine(RE2 port) for a couple of reasons:

1. The initial scope of Secret Detection Service(SDS) was to contain anything and everything pertaining to Secret Detection. That means, pulling the files from a particular source for scanning, updating metadata in the DB, running post-processing actions like revocation, to name a few. Given this idea, we could leverage and integrate with existing systems from GitLab Rails like workers, cache, DB etc. However, we later descoped to make it a stateless service responsible for just scanning the given payloads while Pre and Post-processing steps are handled at GitLab Rails side.
2. There was an active development of SPP feature by vendoring SD Ruby gem so the migration would've pushed the SPP feature release further. Given the strict release timeline we had, we

couldn't have afforded it. We needed a non-disruptive approach.

3. Runway did not (and still doesn't) support deployment to Self-Managed and Dedicated environments. Cloud Connector was an effortful option to consider. Runway had plans to support it so a temporary approach of reusing the on-going work (vendored gem) made more sense.

4. RE2 Regex performance was still sufficiently better in terms latency and memory consumption than built-in Go regex. We did not explore Go's alternative regex engines.

There are some quirks that come with choosing Ruby

1. We over-emphasized on raw regex performance of Ruby RE2 during the decision-making such that we skipped other crucial factors that contribute to the overall performance, particularly when serving the scan via HTTP request. Some of the missed out factors are Server throughput, request latency, memory consumption + GC, Cost-effectiveness due to Resource utilization etc., and Ruby underperforms in all of these factors due to the following:

1. Ruby generally has high-memory footprint, starting from its runtime VM to regex operations, and its GC is not optimised (like any other interpreted languages). We did consider "running the scan within subprocess" approach to address it, however, we had to drop it in SDS since GRPC doesn't support forking processes.

2. Global Interpreter Lock(GIL) allows only one request at a time per-process-per-cpu, keeping the other requests blocked for longer periods that in turn could potentially block the customers on git push operation.

3. No built-in support for concurrency. Even parallelism isn't easily achievable as Ruby doesn't effectively utilize multiple CPU cores and threads due to GIL.

2. Ruby lacks first-class support for GRPC, so it uses C-based GRPC library under the hood. C-Ruby bindings introduces some overhead in addition to missing language-specific optimizations, like for we have for Go.

3. Considering the plan of SDS as the sole platform of running scans across different scan target types, its high throughput and low latency becomes a mandatory requirement. Looking at the metrics of GRPC servers, Ruby is among the poorest performers in the list.

4. Missing first-class support for generating efficient binary executable makes it difficult to unify the Secret Scanning engine across the entire Secret Detection suite of features.

Proposal

The proposal was explore other performant languages and regex engines suitable for running SD scans in different scan modes i.e, as a Distributed service and as an executable binary.

Languages (and engines) considered for the decision

1. Ruby with RE2 Port

2. Go with Vectorscan Port

3. C++ with RE2 (native) and Vectorscan (native).
4. Rust with Std Regex and Std Regex w/ parallelised rule processing.

Conclusion

The final conclusion for the language and regex engine came out to be the combination of Go and Vectorscan (Port).

Results for micro-benchmarking and grpc throughput testing can be found [here](#). The source code and dataset used for testing can be found [here](#).

Why Go will be suitable moving forward?

1. Go has a significant upper hand (around 8-10x) over Ruby in terms of overall performance(mentioned above) such that it could compensate for its moderate regex performance.
2. We did not explore other Go regex alternatives during the Spike since we were convinced about using Ruby due to other constraints mentioned above. There could have been a possibility that a third-party Go regex libraries perform better.
3. First-class support for building executable binary makes Go suitable even in the areas where accessing SD service is not feasible. This also helps us with building a unified scanner engine as we could serve Secret Detection as a Service, Binary executable and CLI with a single Go source code.
4. Having first-class GRPC support, we have better tooling for monitoring, testing, debugging and plethora of middleware extensions. Go also has better alignment with gRPC's type system, so fewer unexpected type-related crashes
5. Given the critical role of SDS in the future, Go's maintainability, dependency management and its type system holds a significant advantage.
6. Adding a cherry on top, integration with Cloud Connector for SDS wouldn't have the blocker of a missing auth SDK, like we currently have for Ruby.

In addition to above, there are more positives of choosing Go over Ruby like efficient GC, low runtime resource consumption, smaller container size leading to faster deployment, efficient file handling, native go-routines w/ channels for a clean concurrency code, compile-time checks, mature profiling/instrumentation tools, cloud native i.e, better suited for containerized and microservice architecture.

Why now is the right time?

The scanner engine is currently stable with no significant feature planned in the immediate roadmap. Secret Detection Service is still in Beta with major features that actively use SDS in the planning phase. We are at an inflection point where delaying this decision further will exponentially increase the migration complexity and technical debt as SDS matures and becomes more deeply integrated. This is our only chance to establish the appropriate technical decision for the long-term.

SECTION: engineering/architecture/design-
documents/secret_detection/decisions/008_unified_scan_engine.md

Context

Currently, the Secret Detection scans are running for different scan target types like Source code (via Pipeline SD), Git commits (Push Protection), Issue description/comments (Client-side SD), and more target types to follow. All the mentioned scan target types have different scan engines (Gitleaks, one managed by the team, another one by Web frontend), and to add complexity to the mix, there are certain teams at GitLab maintaining their version of secret detection. This leads to multiple issues (outlined below) and an inconsistent experience for customers:

It causes a significant burden of maintaining feature parity across multiple engines.
It limits us from optimizing the core scanning logic for performance and efficiency.
It becomes difficult to ensure our ruleset quality (precision and recall) against all the scan engines.

It also becomes difficult to optimize rule patterns which are generally done against a specific regex engine.

Inconsistent integration of intelligent scanning capabilities with the engines based on their compatibility.

Proposal

The proposal is to build a unified Secret Detection core scanning engine that will be used for all the scan target types. The engine's core responsibility will be to run a Secret Detection scan for a given set of payload(s) against the provided scan configuration. The pre-processing (gathering/structuring payload into a scannable format) and the post-processing steps (validity checks etc) will be managed on the caller side. This setup will ensure the reusability of the core scanning logic across all the implementation layers of scan target types.

Characteristics of the unified scan engine

The engine is portable. This is crucial considering some scan target types (source code/artifacts) running in an isolated environment like GitLab CI.

The engine can run transactional or streaming-based scan.

The engine is cross-platform compatible.

The engine is stateless by nature.

The engine is highly resource efficient to be able to run in a resource-constrained environment (like in client IDE or small CI environment).

The engine is to tweak the scan behavior by accepting user inputs (with defaults):

- exclusions (rule/path/value)

- custom ruleset

- ruleset version

- timeout constraints

- payload size limit

- payloads acceptable via file-path/dir-path/stdin (binary executable)

- configurable resource (memory/cpu cores) limit (binary executable)

The engine is able to run as a binary executable in air-gapped environments.

Pre-requisite

The idea of unified engine could hold true only if we have a scan engine that is highly efficient yet portable by nature. As per the decision, we need to have Vectorscan-based Go scan engine implemented.

Mode of using the Scan Engine

The minimalistic scope and the stateless nature of the proposed scan engine will open up the portability advantage, which is a necessity for certain target scan types (source code or job artifacts running in CI env). Therefore, the scan engine could be adopted in one of the following three modes depending on the nature of the scan target type (traffic,size,etc.):

As a Distributed Service: The scan engine will be wrapped with a REST/gRPC layer having scan API endpoints. The caller makes the scan request over the network. This mode is suitable for SD features having high traffic with lightweight payloads (<1MB). Example: Scanning Work Items via Secret Detection Service

As an Embedded module: The core engine in this mode is embedded within the same host as the caller application. The caller invokes the scan for a scannable payload. This mode is transactional by nature. We are already using this mode for the Push Protection feature where the engine is embedded as a Ruby Gem and installed in the Rails monolith. The Rails monolith makes the scan request (including git diff data as a scannable payload) to the gem.

As a Batch processor: This is a special case to support in-storage processing where the Secret Detection program (+engine) runs where the data resides. This reverse approach is suitable for scan target types having larger data sizes, like source code or job artifacts, to avoid data-transfer costs incurred b/w data storage and scan servers. The primary difference when compared to Embedded mode is that the caller includes the scannable payload within the scan request whereas in Batch mode, the caller points at the scannable payload(s) available at the target host (where the program and data reside), e.g. passing a file path along with the request.

Adapters

The engine's minimalistic scope of running the scan for the given payload implies that the caller should implement pre-processing and post-processing steps of the scan. Since the scan target types are located at different sources (ex: Rails/CI/Gitaly), it is important to have a consistent implementation approach across all target types for better maintainability. In addition, there is a decent amount of common logic that exist in the pre-processing and post-processing steps across different scan target types which could be reused between Adapters.

We will follow the concept of Adapters where an Adapter sits between the caller and scan engine abstracting the implementation details, similar to Language Servers for IDE. Adapters are primarily used in Embedded or Batch mode.

Example Scenario: We're currently using gitleaks scan engine in Pipeline SD scan. Though gitleaks offer a lot of functionalities we don't needed, there are three of them needed for this usecase:

1. To traverse through the git commit tree and retrieve diff data for each commit
2. Run secret detection scan on the retrieved content(scannable payload) efficiently (ex: scan parallely, early-exit through keyword-based match, filter noise/FP on the detected findings through entropy etc.)
3. Create a consumable response as requested by the client via scan params

The above process typically the same for any scan target type where the Step 1 represents pre-processing phase of the scan, Step 2 represents the SD Scan operation itself, and finally the Step 3 represents the post-processing phase of the scan. The pre-processing and post-processing logic are generally scan target type dependent whereas the scan logic is common across.

If we replicate the above process keeping the context of Adapters + Unified scan engine in mind, we would have an Adapter (say SourceCodeAdapter) responsible for handling pre-processing and post-processing logic whereas the Scan engine responsible for running the SD scan for the given scannable payloads.

Here's the comprehensive view of unified scan engine with the different Adapters, each for a scan target type.

Distribution

Given there are different modes of using the scan engine and the Vectorscan regex engine being platform-dependent, it is important to identify how the engine is made accessible within GitLab applications.

When using the engine as a distributed (grpc) service: The grpc service will be deployed using Runway making it accessible to internal GitLab.com applications. The clients can make remote scan request to the service via grpc endpoints.

When using the engine as an Embedded module or Batch processing: Depending on the client application, the nature of embedding differs. The engine can be imported as a Go module for Go-based applications whereas for Non Go applications (mostly Ruby in GitLab Rails), there are two choices:

1. Write Ruby C extension for a static library generated from Go's scan engine source
2. Package Go binary executable within GitLab Rails using Omnibus

Considering the VectorScan engine scans on a chunk of data instead of a line-by-line basis (current approach), the high heap usage problem is no longer applicable. In addition, Vectorscan engine operates on a pre-allocated scratch space contributing to the lower memory consumption of the overall scan. The throughput of calling Go methods via C-function adds barely any overhead when compared to IPC between Ruby <-> Go processes. All that said, the decision is leaning towards writing Ruby C extension. However, we are yet to conduct a spike to

confirm this decision with a data-driven evidence.

SECTION: engineering/architecture/design-documents/secret_manager/_index.md

{{< engineering/design-document-header >}}

Summary

GitLab users need a secure and easy-to-use solution to store their sensitive credentials that should be kept confidential ("secret"). GitLab Secrets Manager is the desired system that provides GitLab users to meet that need without having to access third party tools.

Motivation

The current de-facto approach used by many to store a sensitive credential in GitLab is using a Masked Variable or a File Variable.

However, data stored in variables (masked or file variables) can be inadvertently exposed even with masking.

A more secure solution would be to use native integration with external secret managers such as HashiCorp Vault or Azure Key Vault.

Integration with external secret managers requires GitLab to maintain the integration with the third-party products and to assist customers in troubleshooting configuration issues. In addition, customer's engineering teams using these external secret managers may need to maintain these systems themselves, adding to the operational burden.

Having a GitLab native secret manager would provide customers a secure method to store and access secrets without the overhead of third party tools and to leverage the tight integration with other GitLab features.

Goals

Provide GitLab users with a way to:

- Securely store secrets in GitLab
- Use the stored secrets in GitLab components (for example, CI Runner)
- Use the stored secrets in external environments (for example, production infrastructure).
- Manage access to secrets across a root namespace, subgroups and projects.

Use cases

To help design the architecture, we need to understand how users, in their roles, would operate and use the system. Here are significant use case scenarios that can help drive our design decisions:

1. As a Compliance Manager or Security Ops engineer, I want to use dynamic secrets to provide an additional layer of security for my system.
1. As a Compliance Manager or Security Ops engineer, I want to use automatic secret rotation to provide an additional layer of security for my system.
1. As a Compliance Manager or Security Ops engineer, I want to set secrets expiration policies to provide an additional layer of security for my system.
1. As a Developer, I want sensitive credentials to be fully encrypted so I am not accidentally leaking this information in my deliverables.
1. As a Developer, I want to use secrets to store sensitive credentials to avoid storing this information inside of my code.
1. As a GitLab Admin and as a GitLab Group Owner, I want to have the ability to disable secrets management feature altogether if my organization requires the use of a 3rd party tool.
1. As a Compliance Manager or Security Ops engineer, I need to have audit tools which will show me the status of the secrets and their usage so I can identify any suspicious behavior and ensure our security and compliance of our sensitive credentials.
1. As a Compliance Manager, I need to audit log of secrets usage and management to ensure my organization is following compliance policy.
1. As a DevOps Engineer, I want the deployment process to fetch secrets necessary for deployment directly from a secrets manager.
1. As a Customer of a regulated industry, I need a secrets manager with HSM support so I can meet FIPS requirements.
1. As a US Public Sector customer, I need a secrets manager that is FedRAMP certified to meet security requirements.
1. As a Customer with an air-gapped system, I need a secrets managers that can be installed on-prem and managed on-site to meet network requirements.
1. As a Secret Owner, I need to quickly update or rotate the secret in case there has been a vulnerability to ensure the security of my software supply chain.
1. As a Secret Owner, I need the ability to rollback change to a secret in the event of an accidental change.
1. As a Security Engineer, I want to ensure all secrets are properly deleted/destroyed if they are no longer needed.

Non-functional requirements

- Security
- Compliance
- Auditability

Non-goals

This blueprint does not cover the following:

- Secrets created within GitLab to allow external resources to access GitLab personal access tokens, such as personal access tokens.

Decisions

- ADR-005: Non-hierarchical key structure for secrets in OpenBao
- ADR-007: Use OpenBao as the secrets management service
- ADR-008: Redesigning secrets manager without a Rails database table

Superseded

These documents are part of the initial iteration of this blueprint.

- ADR-001: Use envelope encryption
- ADR-002: Use GCP Key Management Service
- ADR-003: Build Secrets Manager in Go
- ADR-004: Stateless Key Management Service
- ADR-006: Use AppRole authentication method between Rails and OpenBao

Proposal

The secrets manager feature will be available on both SaaS and Self-Managed installations and will consist of two core components:

1. GitLab Rails
1. OpenBao Server

mermaid
flowchart LR

```
u([User]) --interacts with--> glab[GitLab Rails]--provisions secrets in-->o{OpenBao}
```

```
glab --starts--> p[Pipeline]
```

```
p --fetches secrets from--> o
```

Because pipelines can be external to GitLab infrastructure, OpenBao thus needs to be open to externally accessible and ACL protected. However, we'll initially restrict the provisioning of secrets and authentication roles through the GitLab Rails UI and limit JWT token creation to pipeline execution, until we can build multi-tenancy extensions into OpenBao.

<details>

<summary> Digression on OpenBao visibility </summary>

Technically there are two approaches to make OpenBao visible:

1. Directly, making the API address listen globally to the public internet.
2. Via GitLab Rails proxying requests on behalf of callers.

In particular, with the latter, while we have the option of replacing the backend secret store with a different provider in the future, we incur additional effort for every dynamic secret provider we support: we can either implement a thin proxy over the OpenBao API (making switching harder as we're still tightly coupling to OpenBao), or we can implement a bespoke translation layer per call.

Outside of a few proprietary vendors offering dynamic secrets, we'd thus likely have to rewrite the functionality from scratch anyways if we were to switch. Thus, we could stick to a vaguely OpenBao/Vault-compatible API design (for clients) but modify it as necessary.

With this trade-off, and given there's interest in expanding past static K/V secrets (into dynamic secrets -- whether for databases, cloud providers, or even GitLab tokens), we suggest using the former approach, exposing OpenBao publicly. This enables non-Pipeline workloads in the future (that simply read secrets, leaving policy creation to GitLab Rails) and lets us use more advanced features such as Transit for other solutions inside GitLab Rails.

</details>

Within OpenBao, we'll use two authentication engines:

1. JWT, to authenticate Rails to OpenBao and created pipeline jobs to OpenBao. All of these JWTs will be issued by GitLab Rails, using GitLab OIDC ID Tokens supported by the existing HashiCorp Vault Runner integration. Claims on tokens for administrative use by